INDIAN INSTITUTE OF TECHNOLOGY MADRAS ZANZIBAR

School of Science and Engineering

Z5007: Programming and Data Structures

M.Tech Data Science & Artificial Intelligence

PROJECT PROPOSAL

Recommendation System

Submitted by:

Student 1: Anurag Roychowdhury ZDA25M004
Email: zda25m004@iitmz.ac.in

Student 2: Sreejita Roy ZDA25M008
Email: zda25m008@iitmz.ac.in

Instructor: Prof. Innocent Nyalala

Submission Date: November 25, 2025

Contents

1 Executive Summary

- The problem we are solving:

  There is a need for an intelligent system that can automatically predict user preferences and recommend relevant content based on past behavior. The challenge lies in analyzing large volumes of user–movie interaction data and identifying meaningful patterns that reflect individual tastes.

- Our approach:

  - Use collaborative filtering by modeling users and items as a graph to capture relationships based on interactions and rating patterns.
  - Apply content-based filtering by extracting TF-IDF vectors from movie descriptions and storing them efficiently using hash tables for quick lookup.
  - Combine scores from both models (hybrid recommender) and use a heap or priority queue to efficiently select the top-N recommendations for each user.
  - Allow real-time updates so new ratings, movies, or users can be added without retraining the entire system.
  - Include cold-start handling by relying more heavily on content-based features when user history is limited.
  - Evaluate overall performance using metrics such as Precision, Recall, and NDCG to measure ranking quality and recommendation relevance.

- Expected outcomes:

  - Deliver personalized recommendations in under 1 second, ensuring that users receive quick suggestions without noticeable delay.
  - Handle 1000+ items and 500+ users efficiently, maintaining performance and responsiveness as the dataset grows, while avoiding slow computations or memory bottlenecks.
  - Provide a scalable, modular system with a clear complexity analysis, allowing future expansion or integration of additional features without major redesign, and making it easier to understand, maintain, and optimize.

2 Team Information

2.1 Team Members

Name	Roll Number	Email
Anurag Roychowdhury	ZDA25M004	zda25m004@iitmz.ac.in
Sreejita Roy	ZDA25M008	zda25m008@iitmz.ac.in

2.2 Team Collaboration Plan

- **Student 1 responsibilities:**
  Implement collaborative filtering, graph structures, evaluation metrics.
- **Student 2 responsibilities:**
  Implement content-based filtering, hash tables, heaps, testing strategy.
- **Shared responsibilities:**
  Hybrid scoring integration, dataset preprocessing, documentation, presentation.

- **Communication plan:**
  We will have weekly sync meetings during our scheduled lab sessions to discuss progress, challenges, and next steps.
  For ongoing communication and quick updates between meetings, we will use tools such as email, messaging apps, or a shared project workspace.

3 Project Selection

3.1 Chosen Option

Project Option: Recommendation System

Specific Algorithm/System: Hybrid Movie/Series Recommendation System

3.2 Motivation

- Traditional recommendation systems often rely on either collaborative filtering or content-based methods, which struggle with cold-start problems and lack personalization depth.
- Our goal is to build a hybrid recommendation engine that intelligently combines both approaches to deliver accurate, personalized suggestions for movies and series.
- By leveraging graphs for user–item relationships, hash tables for fast metadata access, and heaps for efficient ranking, our system will provide recommendations in under one second.
- This project aligns with our passion for data-driven storytelling and our career goals in AI-powered personalization. It also reflects the growing demand for smarter, scalable recommendation systems in the entertainment industry.

4 Problem Statement

4.1 Problem Description

Current recommendation systems often rely solely on collaborative filtering or content-based methods, each with limitations. Collaborative filtering struggles with cold-start problems, while content-based filtering may lack diversity. Our proposed hybrid system integrates both approaches, using efficient data structures to ensure scalability and speed. This system will provide personalized recommendations for users while maintaining performance under large datasets.

4.2 Scope

In Scope:

Feature 1: Collaborative filtering (user-based, item-based)

Feature 2: Content-based filtering (TF-IDF, inverted index)

Feature 3: Hybrid scoring with weighted combination

Feature 4: Cold-start handling

Feature 5: Evaluation metrics (precision, recall, NDCG)

Feature 6: Complexity analysis

Out of Scope:

Feature 1: Deep learning-based recommenders

Feature 2: Real-time deployment on production servers

5 Technical Approach

5.1 Data Structures

- Graph (Adjacency Lists): Represent user–item relationships for collaborative filtering. Supports traversal and similarity computation.
- Hash Tables (Dictionaries): Store item features, user profiles, and similarity caches for fast lookups.
- Heap/Priority Queue: Efficiently maintain top-N recommendations during ranking.
- Supporting data structures: List / Array – To store ratings, features, items; Set – Track unique items / users, avoid duplicates; Queue – For real time updates or streaming recommendations.

5.2 Algorithms*:*

1. **Collaborative Filtering (user-based similarity):**
   - Purpose: Identify similar users based on their ratings and recommended items liked by those users.
   - Input: User-item rating matrix, target user ID
   - Output: List of recommended items for the target user.
   - Expected time complexity:
     Similarity Computation: O(U*I) where U: number of users and I: Number of items(movies/series)
     Recommendation Generation: O(K*I) where K: number of neighbors

- Expected space complexity: O (U + I) for storing adjacent lists and similarity cache

2. **Collaborative Filtering (Item based similarity):**
   - Purpose: Recommend items similar to those the user has already liked.
   - Input: item-user rating matrix, target user ID
   - Output: Ranked list of similar items
   - Expected time complexity:
     Similarity Computation: O(I*I) for all item pairs, optimized with pruning
     Recommendation Generation: O(K*I)
   - Expected space complexity: O (I + U) for storing item vectors and similarity cache
3. **Content-Based Filtering (TF-IDF + Cosine Similarity):**
   - Purpose: Recommend items based on similarity of content features
   - Input: item metadata (text), user profile (aggregated TF-IDF vector)
   - Output: Ranked list of content-similar items
   - Expected time complexity: TF-IDF vectorization: O (N*T) where N is the number of items and T is the number of terms
     Cosine Similarity: O (M*T) where M is the number of candidate items
   - Expected space complexity: O (N*T) for storing sparse TF-IDF matrix and inverted index
4. **Hybrid scoring (Weighted Combination):**
   - Purpose: Combine collaborative and content-based scores to produce final recommendation scores.
   - Input: scores from CF (user/item) and content-based filtering, weight parameters α, β, γ.
   - Output: Final score for each candidate item.
   - Expected time complexity: O(M) where M is the number of candidate items
   - Expected space complexity: O(M) for storing final scores
5. **Top-N Ranking (Heap/Priority Queue):**
   - Purpose: Efficiently select top-N items based on hybrid scores.
   - Input: List of scored items, N (number of recommendations)
   - Output: Top-N recommended items.
   - Expected time complexity: O (M log N) where M is the number of candidates
   - Expected space complexity: O (N) for the heap
6. **Cold-Start Handling (Content-Based Fallback):**
   - Purpose: Provide recommendations for new users or items using metadata alone.
   - Input: User preferences or item metadata.
   - Output: Initial recommendations based on content similarity.
   - Expected time complexity: O (M * T) similar to content-based filtering
   - Expected space complexity: O (N * T) for TF_IDF matrix and inverted matrix

5.3 System Architecture

- Main components/modules:

  Data Preprocessing

  Collaborative Filtering Engine

  Content-Based Engine

  Hybrid Scoring Module

  Evaluation Module

  User Interface (CLI/Notebook)

  Data flows from dataset

- How they interact:

  Data flows from dataset → preprocessing → CF/CB engines → hybrid scoring → ranking → evaluation.

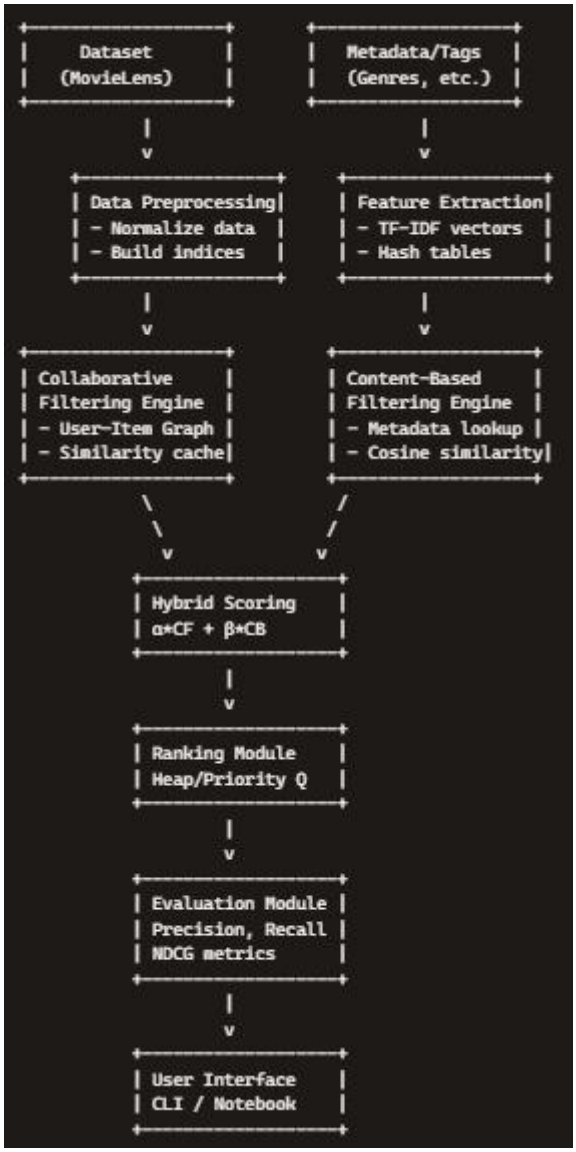


Figure 1: System Architecture Diagram

Description of system architecture:

The system begins with data preprocessing, where raw ratings and metadata are normalized and transformed into usable structures. The Collaborative Filtering Engine leverages a user–item graph to compute similarity-based recommendations, while the Content-Based Engine uses hash tables and TF–IDF vectors to recommend items based on metadata. These outputs are combined in the Hybrid Scoring Module, which balances collaborative and content-based signals. The Ranking Module uses a heap to efficiently select the top-N items. Finally, the Evaluation Module validates recommendations against baseline models, and results are presented through a CLI/Notebook interface for easy interaction.

6 Dataset

6.1 Dataset Selection

Dataset: MovieLens 1M Dataset

Source - https://grouplens.org/datasets/movielens/

Size: 1 million ratings, 6,000 users, 4,000 movies

6.2 Dataset Description

Property	Value
Number of samples	1,000,000 ratings
Number of features	20+
File format	CSV
File size	20 MB
Target variable	User ratings (1-5 scale)

Features:

Feature 1: User ID (integer)

Feature 2: Movie/Series ID (integer)

Feature 3: Rating (float)

Feature 4: Timestamp (integer)

6.3 Data Preprocessing

- Normalize ratings
- Tokenize and vectorize movie metadata (TF-IDF)
- Build inverted index for terms
- Split dataset into train/test (80/20)

7 Implementation Plan

7.1 Week-by-Week Timeline

Week	Tasks
Week 6	Submit proposal, setup development environment
Week 7	Implement core data structure 1, load and preprocess data
Week 8	Implement core data structure 2, begin algorithm implementation
Week 9	Complete algorithm implementation, basic testing
Week 10	Submit progress report, refine implementation
Week 11	Performance optimization, comprehensive testing
Week 12	Benchmarking and comparison, begin documentation
Week 13	Complete documentation, create presentation
Week 14	Final testing, submit final project, presentation

7.2 Development Tools

Programming Language: Python 3.8+

IDE: VS Code

Libraries:

- numpy (for numerical operations)
- pandas (for data manipulation)
- matplotlib (for visualization)
- Scikit-learn

Version Control: GitHub (private repository)

7.3 Testing Strategy

1. **Unit Testing:** Test individual components such as:

   Graph construction (user–item adjacency lists).

   Hash table lookups for item features and similarity caches.

   Heap operations for top-N ranking.

2. **Integration Testing:** Test components together - Validate that cold-start handling integrates seamlessly with hybrid scoring.

3. **Performance Testing:** Test on large datasets
   Run the system on the MovieLens dataset (≥1000 items, ≥500 users).
   Measure runtime for generating recommendations per user, ensuring results are produced in under 1 second.

4. **Validation:** Compare with baseline/sklearn

8 Expected Challenges

8.1 Challenge 1: Cold-start problem

Description: New users without prior ratings may receive poor or irrelevant recommendations. Newly added movies/series may not be suggested to users until sufficient interaction data is collected.

Impact: This reduces user engagement and trust in the system during initial use.

Mitigation Strategy: Use content-based fallback - When collaborative filtering cannot generate recommendations (due to lack of data), the system "falls back" to content-based filtering.

8.2 Challenge 2: Scalability

Description: As the dataset grows (thousands of users and items), recommendation generation may slow down.

Impact: Memory usage could increase significantly if inefficient structures are used. Without pruning and optimization, the system may fail to deliver recommendations within the required <1 second runtime, affecting usability.

Mitigation Strategy: Use efficient data structures and pruning.

8.3 Challenge 3: Balancing weights in hybrid scoring

Description: Incorrect weight distribution between collaborative and content-based components may bias recommendations.

Impact: Over-reliance on one method could reduce diversity or personalization quality.

Mitigation Strategy: Fine-tune via cross validation techniques.

9 Success Criteria

9.1 Functional Requirements

System must:

- Requirement 1: Generate top-N recommendations in <1 second
- Requirement 2: Support new user/item addition
- Requirement 3: Provide evaluation metrics

9.2 Performance Metrics

Metric	Target	Minimum Acceptable
Precision	>75%	65%
Recall	>70%	60%
NDCG	>80%	70%
Runtime	< 1 sec	< 5 sec

9.3 Comparison Baseline

- Baseline 1: sklearn implementation
- Baseline 2: Naïve popularity-based recommender
- Success if: Success if hybrid system outperforms baselines by ≥10% in NDCG

10 References

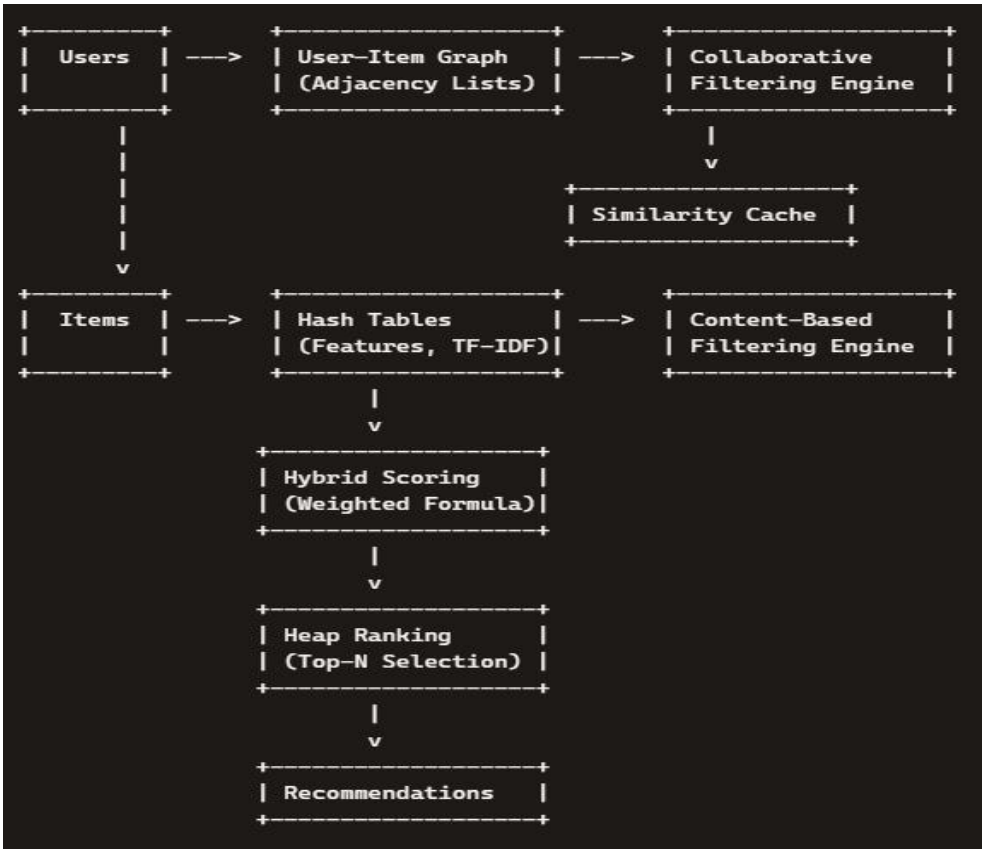
1. Reference 1: GroupLens MovieLens Dataset Documentation
2. Reference 2: GeeksforGeeks – Recommendation System in Python

11 Appendices

11.1 Appendix A: Sample Data

User ID	MovieID	Rating	Timestamp
1	1193	5	978300760
1	661	3	978302109

11.2 Appendix B: Preliminary Diagrams



End of Proposal

Submitted to: Prof. Innocent Nyalala

Date: November 25, 2025